

ExoNet

Classifying Kepler exoplanet candidates

Trilok - Group 01 (solo)

ITAI 2372 Final Project, Implementation Track

April 2026

The problem

Kepler watched 150,000+ stars looking for transits.

It produced thousands of transit-like signals.

Each one needs to be vetted by hand: real planet, instrument noise, or eclipsing binary?

Vetting is slow expert work.

TESS keeps adding more candidates to the queue.

Question: can a small ML model do a good first-pass triage?

What I built

A supervised classifier:

- Input: 10 features per candidate (orbit, planet, host star).
- Output: CONFIRMED or FALSE POSITIVE, with a probability.

Delivered as a small Python CLI:

- `python -m src.main train`
- `python -m src.main predict --koi-period ... (etc)`
- `pytest`

Data

Source: NASA Exoplanet Archive, Kepler KOI cumulative table.

- Initial size: about 9,500 KOIs.
- After dropping CANDIDATE rows (label undecided): ~7,500 rows.
- Roughly balanced between CONFIRMED and FALSE POSITIVE.

Cached locally on first download.

Synthetic-data fallback if the archive is unreachable.

The 10 features

Orbit

- koi_period (days), koi_duration (hr), koi_depth (ppm)

Candidate planet

- koi_prad (Earth radii), koi_teq (K), koi_insol (Earth flux), koi_model_snr

Host star

- koi_steff (K), koi_slogg (log g), koi_srad (solar radii)

Picked these because (1) they are dense in the dataset and (2) they map to what astronomers actually look at.

Pipeline

1. Load CSV from NASA archive (or cached copy).
2. Coerce features to numeric, drop empty rows, median-impute NaNs.
3. Drop CANDIDATE rows. Map labels to 0 / 1.
4. Stratified 80 / 20 train / test split (seed = 42).
5. Fit RandomForestClassifier (300 trees, balanced weights).
6. Score on test set, save model + metrics + confusion matrix.

Why Random Forest

Considered a small neural net first.

Decided against it because:

- Dataset is small (~7,500 rows) and tabular. Trees usually win at this scale.
- Random Forest does not need feature scaling.
- Gives feature importances I can talk about (coming up next).
- Way easier to defend in a presentation.

Future work: try XGBoost on the same split as a comparison.

Results on real Kepler data

Test set size: 1,518 rows

Accuracy 0.926

Precision 0.883

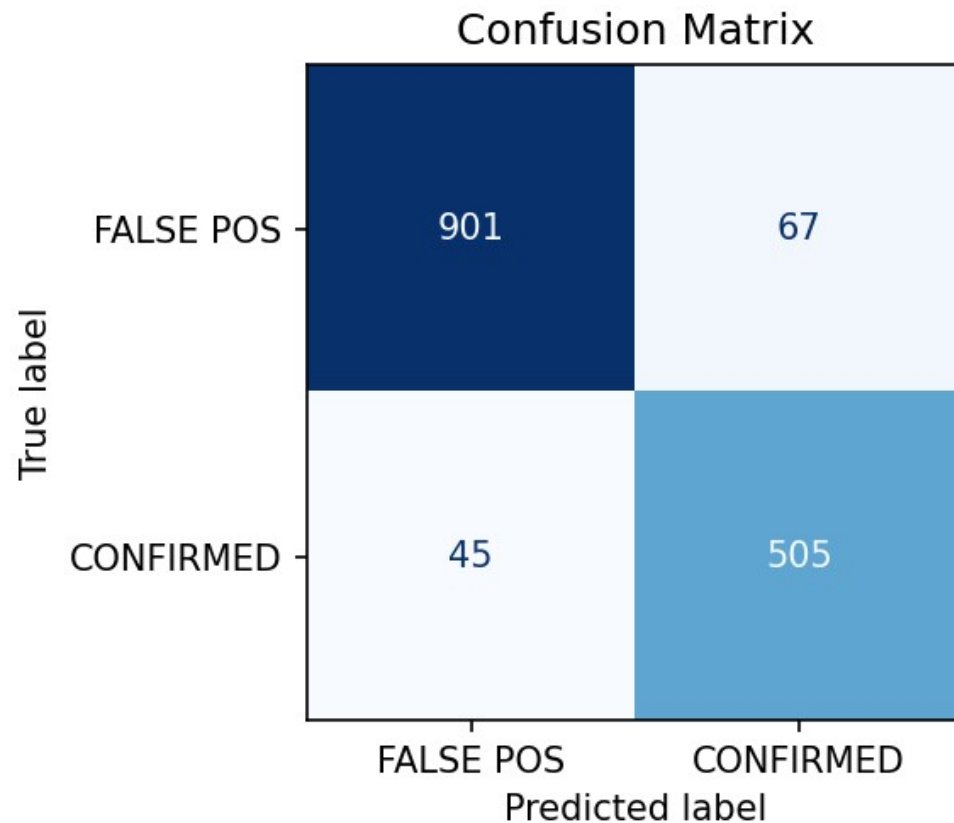
Recall 0.918

F1 0.900

ROC-AUC 0.976

Beat the success criteria from the proposal (≥ 0.85 acc, ≥ 0.92 ROC-AUC).

Confusion matrix



Most errors are false positives flagged as confirmed (67 cases).

Could lower the decision threshold if precision matters more than recall for the use case.

Top features by importance

1. koi_prad (planetary radius) 0.231
2. koi_model_snr (transit signal-to-noise) 0.149
3. koi_period (orbital period) 0.128
4. koi_depth (transit depth, ppm) 0.101
5. koi_insol (insolation flux) 0.097

These line up with what astronomers actually look at when triaging candidates by hand. That is a sanity check that the model has learned something reasonable, not a shortcut.

Limitations and future work

What this project does not do (yet):

- Only a single train/test split. K-fold CV would give better confidence intervals.
- Only a Random Forest baseline. Worth comparing to XGBoost or LightGBM.
- Trained only on Kepler data. TESS uses similar features and would be the obvious next dataset.
- No web UI or hosted demo. Just a CLI for now.

Thanks! Questions?

Repo: FP_GROUP01_Trilok_ITAI2372

Run it: `python -m src.main train`

Data: NASA Exoplanet Archive, Kepler KOI cumulative table